Maestría en
Software

Patrones de Diseño de Software

Autor(es): Mauricio Ortiz Ochoa
mortizo@ups.edu.ec

Tema:

Unidad 01. Patrones de diseño creacionales

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Tema:

Propósito y organización de la asignatura

- *Propósito y organización de la asignatura*
- Fundamentos de patrones
- Bases de POO para comprender patrones
- Actividad diagnóstica

Este material es de autoría y uso académico exclusivo de Mauricio Ortiz Ochoa

Propósito y organización de la asignatura

Objetivos de la asignatura

- Comprender los fundamentos y la clasificación de los patrones de diseño.
- Seleccionar patrones según el problema y el contexto.
- Modelar e implementar soluciones orientadas a objetos.
- Utilizar IA como apoyo, validando críticamente sus resultados.

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Propósito y organización de la asignatura

Resultados de aprendizaje

- **RA1**. Selecciona y aplica patrones creacionales para resolver problemas de creación de objetos, revisando críticamente las alternativas elaboradas con apoyo de inteligencia artificial.
- **RA2**. Modela soluciones de software aplicando patrones estructurales de acuerdo con el contexto y las necesidades de diseño.
- **RA3**. Implementa soluciones aplicando patrones de comportamiento para organizar responsabilidades e interacciones entre objetos.

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Propósito y organización de la asignatura

Contenidos mínimos

Unidad 01. Patrones de diseño creacionales

- Fundamentos de patrones.
- Principios de programación orientada a objetos.
- Código limpio.
- Factory Method.
- Builder.
- Singleton.
- Principios SOLID.
- Uso crítico de IA como apoyo al análisis, implementación y revisión.

Unidad 02. Patrones de diseño estructurales

- Composite
- Adapter
- Bridge
- Facade

Unidad 03. Patrones de diseño de comportamiento

- Strategy
- Template Method
- State
- Observer

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Propósito y organización de la asignatura

Cronograma de clases: semana 1

	jueves, 01 de octubre de 2026
18H00	Propósito y organización de la asignatura
18H30	Patrones en diferentes dominios
18H50	Concepto y estructura de un patrón
19H20	Patrones de diseño GoF
19H50	RECESO
20H10	Abstracción, objetos y clases
20H30	Relaciones entre clases
20H50	Dependencia, encapsulamiento y herencia
21H10	Actividad diagnóstica
21H50	Retroalimentación y cierre

	viernes, 02 de octubre de 2026
18H00	Diagramas de clases
18H50	Composición
19H00	Agregación
19H20	Actividad en clase: diagrama completo
19H50	RECESO
20H10	Resolución de la actividad en clase
20H30	Código limpio y deuda técnica
21H00	Legibilidad
21H20	Diseño mantenible
21H50	Retroalimentación y cierre

	sábado, 03 de octubre de 2026
08H00	Patrones creacionales
08H30	Factory Method
08H50	Revisión en grupo: Factory Method
09H20	Resolución guiada: Factory Method
09H50	RECESO 1
10H10	Builder
10H30	Revisión en grupo: Builder
10H50	Resolución guiada: Builder
11H20	Actividad en clase: Builder
11H50	RECESO 2
12H10	Singleton
12H30	Revisión en grupo: Singleton
12H50	Resolución guiada: Singleton
13H10	Explicación de la Actividad 01 (15p)
13H20	Actividad en clase: Singleton
13H50	Retroalimentación y cierre

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Propósito y organización de la asignatura

Cronograma de clases: semana 2

	jueves, 15 de octubre de 2026
18H00	Abstract Factory
18H20	Revisión en grupo: Abstract Factory
18H50	Resolución guiada: Abstract Factory
19H20	Actividad en clase: Abstract Factory | Revisión grupal de la Actividad 01
19H50	RECESO
20H10	Prototype
20H30	Revisión en grupo: Prototype
20H50	Resolución guiada: Prototype
21H20	Actividad en clase: Prototype | Revisión grupal de la Actividad 01
21H50	Retroalimentación y cierre

	viernes, 16 de octubre de 2026
18H00	Evaluación 01 (20p)
18H50	Code Smells y principios STUPID
19H00	Principio de responsabilidad única — SRP
19H20	Actividad en clase: SRP
19H50	RECESO
20H10	Principio abierto/cerrado — OCP
20H30	Principio de sustitución de Liskov — LSP
21H00	Principio de segregación de interfaces — ISP
21H20	Principio de inversión de dependencia — DIP
21H50	Retroalimentación y cierre

	sábado, 17 de octubre de 2026
08H00	Patrones estructurales
08H30	Composite
08H50	Revisión en grupo: Composite
09H20	Resolución guiada: Composite
09H50	RECESO 1
10H10	Adapter
10H30	Revisión en grupo: Adapter
10H50	Resolución guiada: Adapter
11H20	Actividad en clase: Adapter
11H50	RECESO 2
12H10	Facade
12H30	Revisión en grupo: Facade
12H50	Resolución guiada: Facade
13H10	Explicación de la Actividad 02 (15p)
13H20	Actividad en clase: Facade
13H50	Retroalimentación y cierre

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Propósito y organización de la asignatura

Cronograma de clases: semana 3

	jueves, 22 de octubre de 2026
18H00	Bridge
18H20	Revisión en grupo: Bridge
18H50	Resolución guiada: Bridge
19H20	Actividad en clase: Bridge | Revisión grupal de la Actividad 02
19H50	RECESO
20H10	Decorator
20H30	Revisión en grupo: Decorator
20H50	Resolución guiada: Decorator
21H20	Actividad en clase: Decorator | Revisión grupal de la Actividad 02
21H50	Retroalimentación y cierre

	viernes, 23 de octubre de 2026
18H00	Evaluación 02 (20p)
18H50	Patrones de comportamiento
19H00	Strategy
19H20	Revisión en grupo: Strategy
19H50	RECESO
20H10	Template Method
20H30	Revisión en grupo: Template Method
20H50	Resolución guiada: Template Method
21H20	Actividad en clase: Template Method
21H50	Retroalimentación y cierre

	sábado, 24 de octubre de 2026
08H00	State
08H20	Revisión en grupo: State
08H50	Resolución guiada: State
09H20	Actividad en clase: State
09H50	RECESO
10H10	Observer
10H30	Revisión en grupo: Observer
10H50	Resolución guiada: Observer
11H10	Explicación de la Actividad 03 (30p)
11H20	Actividad en clase: Observer
11H50	Retroalimentación y cierre

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Tema:

Fundamentos de patrones

- Propósito y organización de la asignatura
- *Fundamentos de patrones*
- Bases de POO para comprender patrones
- Actividad diagnóstica

Este material es de autoría y uso académico exclusivo de Mauricio Ortiz Ochoa

Patrones que salvan vidas

- **Problema**: la energía de impacto vertical suele dañar la columna de los tripulantes de helicópteros.
- **Objetivo**: reducir la energía del impacto.
- **Solución**: incorporar bolsas inflables para absorber parte de la energía del impacto.
- **Consecuencia**: una vez desplegadas, afectan la aerodinámica y la operación de la aeronave.



Este material es de autoría y uso académico exclusivo de Mauricio Ortiz Ochoa

Fundamentos de patrones

Patrones que salvan vidas

- **Problema:** el vehículo de exploración debe soportar el impacto contra la superficie de Marte.
- **Objetivo:** amortiguar el impacto y proteger sus componentes.
- **Solución:** utilizar un sistema de bolsas inflables interconectadas.
- **Consecuencia:** el sistema añade masa y volumen, puede producir rebotes y debe retraerse después del aterrizaje.



Este material es de autoría y uso académico exclusivo de Mauricio Ortiz Ochoa

Patrones que salvan vidas

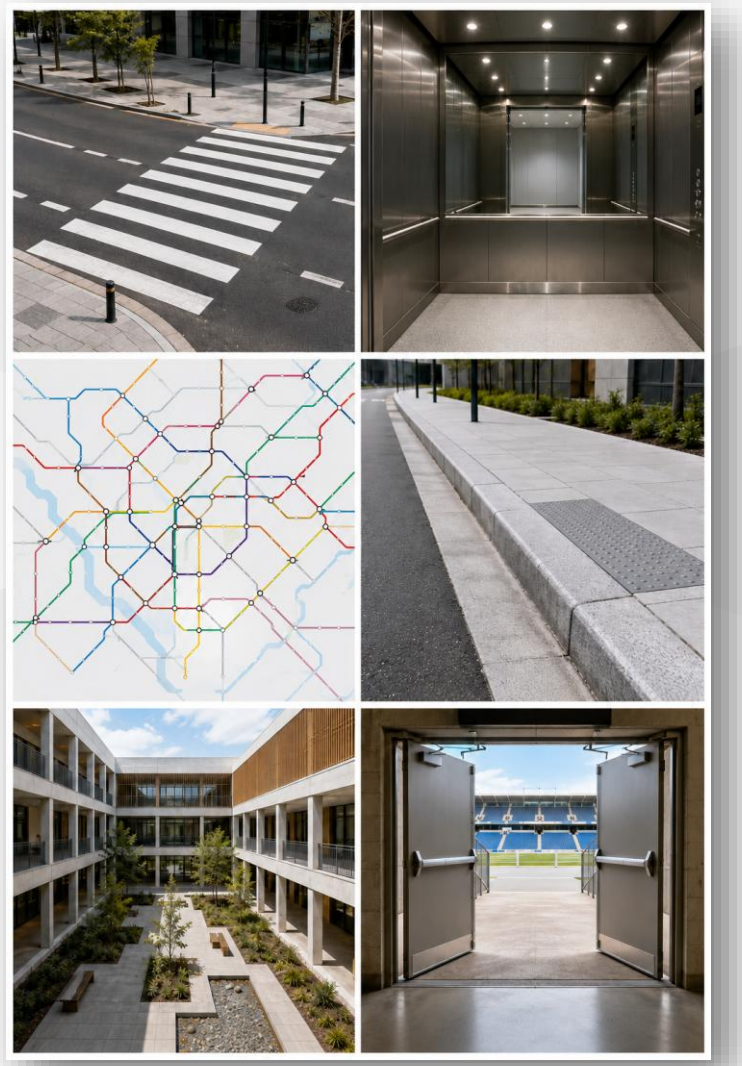
- **Problema:** en un choque frontal, la cabeza del conductor puede impactar contra el volante o la columna de dirección.
- **Objetivo:** amortiguar el impacto y reducir las fuerzas sobre el ocupante.
- **Solución:** air-bag.
- **Consecuencia:** después del despliegue, el sistema debe inspeccionarse y reemplazarse antes de devolver el vehículo a condiciones seguras de operación.



Este material es de autoría y uso académico exclusivo de Mauricio Ortiz Ochoa

Origen del lenguaje de patrones

- A Pattern Language: Towns, Buildings, Construction (1977)
- Autores: Christopher Alexander, Sara Ishikawa y Murray Silverstein, con Max Jacobson, Ingrid Fiksdahl-King y Shlomo Angel.
- “Cada patrón describe un problema recurrente y el núcleo de una solución que puede utilizarse millones de veces, sin hacerlo nunca exactamente de la misma manera”.
- Estructura básica de un patrón:
  - Contexto
  - Problema
  - Solución
  - Consecuencias



Este material es de autoría y uso académico exclusivo de Mauricio Ortiz Ochoa

Fundamentos de patrones

Patrones

¿Qué es un patrón?

- Una descripción reutilizable de una solución general para un problema recurrente en un contexto determinado

¿Qué elementos describe un patrón?

- El contexto en el que aparece el problema.
- El problema recurrente que debe resolverse.
- Las fuerzas o restricciones que deben equilibrarse.
- La estructura esencial de una solución.
- Las consecuencias y compromisos de aplicarla.

¿Qué NO es un patrón?

- Una solución universal.
- Una receta rígida.
- Código listo para copiar.

Un patrón conserva su esencia, pero su implementación cambia según el contexto.

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

¿Qué caracteriza a un buen patrón?

- Resuelve un problema recurrente.
- Se basa en una solución probada en distintos casos.
- Aporta una solución que no resulta evidente para quien enfrenta el problema por primera vez.
- Explica la relación entre el contexto, las fuerzas y la solución.
- Considera su impacto en las personas que lo aplican y utilizan.

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

De la arquitectura al diseño de software detallado

- 1977 — Origen en la arquitectura
  - Christopher Alexander y sus colaboradores publican A Pattern Language, una colección de soluciones recurrentes para problemas de arquitectura y urbanismo.
- 1987 — Primeros patrones de software
  - Kent Beck y Ward Cunningham presentan en OOPSLA una propuesta para aplicar las ideas de Christopher Alexander al diseño de software orientado a objetos.
- 1994 — Consolidación de los patrones GoF
  - Erich Gamma, Richard Helm, Ralph Johnson y John Vlissides publican Design Patterns: Elements of Reusable Object-Oriented Software, con un catálogo de 23 patrones.
- 1994 — Comunidad de patrones
  - Se celebra la primera conferencia Pattern Languages of Programs (PLoP), dedicada al intercambio y revisión de patrones.

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Fundamentos de patrones

Gang of Four (GoF)

- Ralph Johnson · Erich Gamma · Richard Helm · John Vlissides
- Design Patterns: Elements of Reusable Object-Oriented Software (1994).
- Catalogaron 23 patrones de diseño orientado a objetos.
- Su trabajo popularizó un vocabulario común para comunicar soluciones de diseño recurrentes.



Este material es de autoría y uso académico exclusivo de Mauricio Ortiz Ochoa

Las cuatro partes esenciales según GoF

Según GoF, un patrón de diseño se documenta mediante cuatro elementos:

- **Nombre**: identifica el patrón y facilita la comunicación entre diseñadores.
- **Problema**: explica el contexto, las fuerzas (necesidades y restricciones que deben equilibrarse) involucradas y cuándo resulta aplicable.
- **Solución**: describe los elementos, relaciones, responsabilidades y colaboraciones; no establece una implementación concreta.
- **Consecuencias**: presenta los beneficios, limitaciones y compromisos derivados de su aplicación.

Un patrón reutiliza experiencia de diseño, no código listo para copiar.

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Fundamentos de patrones

Plantilla de documentación de un patrón GoF

Campo	Descripción
Nombre y clasificación	Identifica el patrón y lo clasifica según su propósito y alcance.
Propósito	Resume qué hace el patrón, su finalidad y el problema de diseño que aborda.
También conocido como	Registra otros nombres utilizados para referirse al mismo patrón.
Motivación	Presenta un escenario que explica el problema y cómo la solución lo resuelve.
Aplicabilidad	Indica cuándo utilizar el patrón y cómo reconocer las situaciones apropiadas.
Estructura	Representa gráficamente las clases, los objetos y sus relaciones.
Participantes	Describe las clases u objetos involucrados y sus responsabilidades.
Colaboraciones	Explica cómo interactúan los participantes para cumplir sus responsabilidades.
Consecuencias	Expone resultados, beneficios, limitaciones y compromisos de la solución.
Implementación	Detalla técnicas, decisiones y dificultades relevantes para implementar el patrón.
Código de ejemplo	Ilustra una posible implementación en un lenguaje de programación.
Usos conocidos	Muestra aplicaciones del patrón identificadas en sistemas reales.
Patrones relacionados	Señala patrones semejantes, complementarios o utilizados como alternativas.

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Valor y límites de los patrones

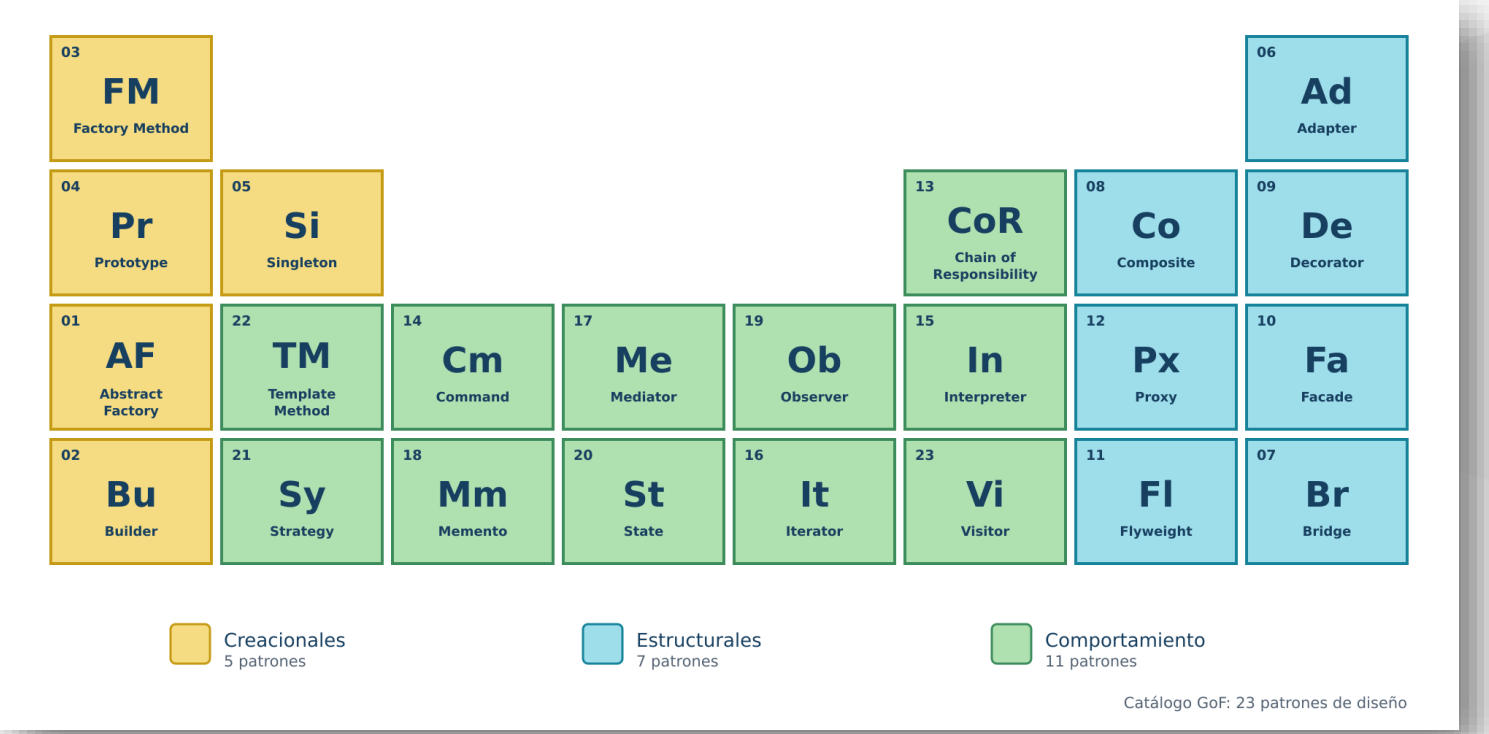
- Capturan y transmiten experiencia acumulada de diseño.
- Proporcionan un vocabulario común para comunicar decisiones de diseño.
- Ayudan a reconocer y resolver problemas recurrentes sin comenzar desde cero.
- Hacen explícitas la estructura, las responsabilidades y las consecuencias de una solución.
- Pueden favorecer la flexibilidad, mantenibilidad y consistencia del diseño cuando se aplican en el contexto adecuado.

Advertencia: un patrón no garantiza software de calidad. Aplicarlo sin un problema real puede añadir complejidad y generar sobrediseño.

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Clasificación de los patrones GoF por propósito

- **Patrones creacionales (5)**: Abstraen el proceso de creación de objetos y reducen la dependencia respecto de clases concretas.
- **Patrones estructurales (7)**: Definen cómo combinar clases y objetos para construir estructuras mayores, flexibles y mantenibles.
- **Patrones de comportamiento (11)**: Definen cómo los objetos se comunican, colaboran y distribuyen responsabilidades para realizar una tarea.



Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Tema:

Bases de POO para comprender patrones

- Propósito y organización de la asignatura
- Fundamentos de patrones
- *Bases de POO para comprender patrones*
- Actividad diagnóstica

Este material es de autoría y uso académico exclusivo de Mauricio Ortiz Ochoa

Bases de POO para comprender patrones

Paradigma y Objeto

- **Paradigma**

Modela un sistema mediante objetos que poseen estado, comportamiento e identidad, y que colaboran para cumplir responsabilidades.

- **Objeto**

Entidad concreta o conceptual que resulta relevante dentro del sistema.

- Juana, María, Ana, Victoria, Mauricio.
- 2024 Kia Rio R, 2027 Chevrolet Spark , 2018 Toyota Yaris.
- 2026 Yamaha 401, 2022 Benelli 1000, 2024 Daytona 250.
- Macbook Air M5, HP 14 CF, Lenovo Ideapad Gaming 5.
- Reebok 8 1/2, Nike 9, Adidas tennis 8.

Este material es de autoría y uso académico exclusivo de Mauricio Ortiz Ochoa

Bases de POO para comprender patrones

Abstracción

Abstraer consiste en seleccionar las características relevantes de un objeto para un contexto e ignorar temporalmente las demás

- Depende del problema que se desea resolver.
- Un mismo objeto puede originar diferentes abstracciones.
- No significa eliminar detalles, sino decidir cuáles importan.



Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Bases de POO para comprender patrones

Clase

Clasificación de un conjunto de objetos con características y funcionalidades similares.

Plantilla para la creación de objetos según un modelo predefinido.

Define la estructura y el comportamiento común de un tipo de objeto.

Estructura

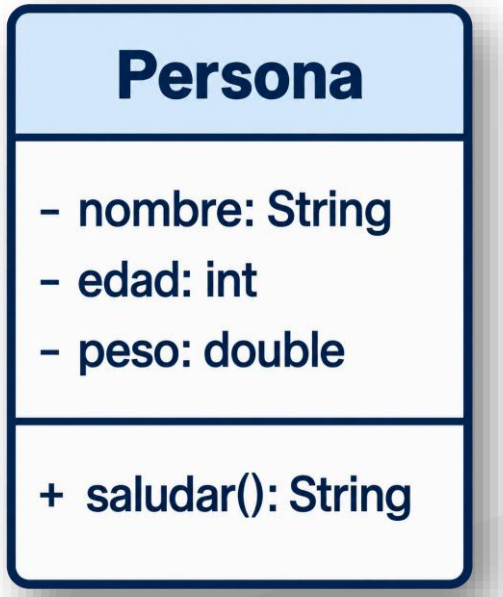
- Nombre: identifica el concepto representado.
- Atributos: describen el estado de sus objetos.
- Métodos: Las funcionalidades del conjunto de objetos similares.



Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Bases de POO para comprender patrones

De la clase UML al código




```java
public class Persona {

    private String nombre;
    private int edad;
    private double peso;

    public String saludar() {
        return "Hola, soy " + nombre;
    }
}
```


Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Asociaciones (relaciones)

- Una asociación representa una relación estructural entre clases cuyos objetos necesitan conocerse o colaborar.
- Puede expresar:
  - Nombre o propósito de la relación.
  - Multiplicidad en cada extremo.
  - Navegabilidad.
  - Roles de las clases participantes.



Este material es de autoría y uso académico exclusivo de Mauricio Ortiz Ochoa

Multiplicidad

- La multiplicidad indica cuántas instancias de una clase pueden relacionarse con una instancia de la otra.
- Se pueden representar varios tipos de multiplicidad.
  - 1: exactamente una.
  - 0..1: ninguna o una.
  - * o 0..*: ninguna o varias.
  - 1..*: una o varias.
  - m..n: entre un mínimo y un máximo.
- La multiplicidad expresa una regla del dominio, no una decisión meramente gráfica.

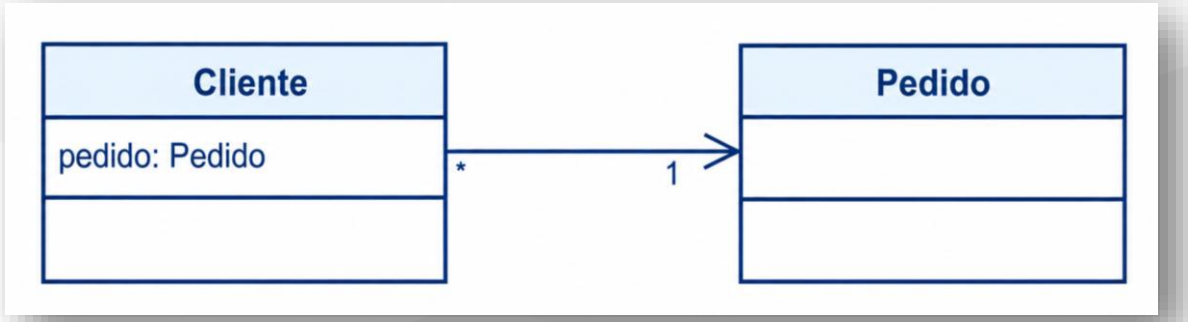


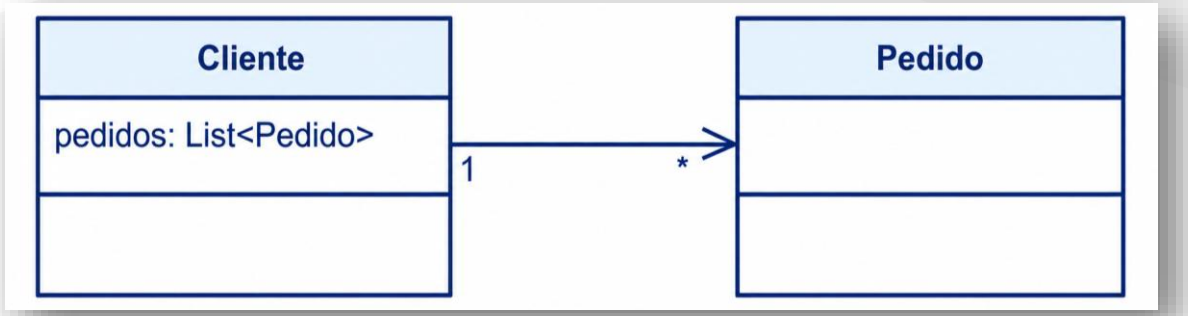
Este material es de autoría y uso académico exclusivo de Mauricio Ortiz Ochoa

Bases de POO para comprender patrones

Navegabilidad

- Esta característica es opcional pero muy importante dentro de las relaciones, indica que una clase conoce o puede acceder a objetos de otra clase.
- La flecha apunta hacia la clase conocida.
- Cuando dos clases se encuentran relacionadas, y lo que se desea es representar que en la ClaseA se va a crear un objeto de la ClaseB, entonces la navegabilidad se representa con una flecha en la ClaseB.
- En la relación se puede describir el nombre del objeto que se creará a partir de una ClaseB en la ClaseA.

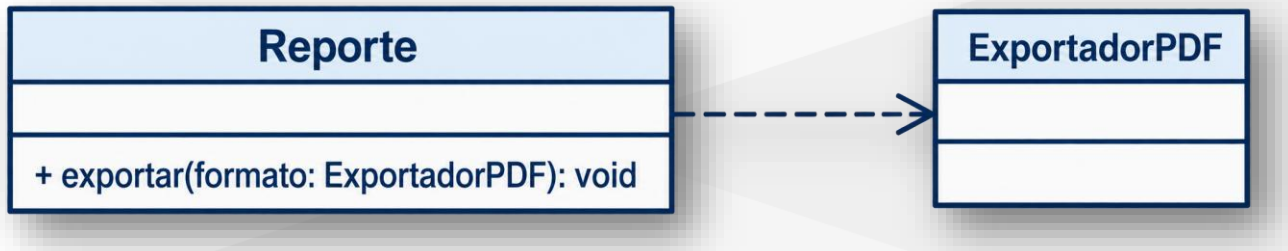




Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Dependencia

- Una dependencia aparece cuando una clase utiliza temporalmente a otra para realizar una operación.
- A diferencia de la asociación, la dependencia no representa necesariamente un vínculo permanente entre los objetos
- En una asociación navegable, la clase suele mantener una referencia como atributo. En una dependencia, la otra clase se utiliza temporalmente como:
    - Parámetro de un método.
    - Variable local.
    - Tipo de retorno.
    - Objeto creado dentro de una operación.



Este material es de autoría y uso académico exclusivo de Mauricio Ortiz Ochoa

Bases de POO para comprender patrones

Visibilidad

- public (+) Permite que el atributo o método sea accesible desde cualquier clase.
- private (-) El atributo o método solo es accesible desde la propia clase.
- protected (#) El atributo o método es accesible desde la propia clase y sus subclases.
- Sin ninguna palabra reservada significa que es accesible desde cualquier clase dentro del paquete.
- Como regla general, los atributos deben mantenerse privados y exponerse mediante operaciones controladas.

```
public int edad;
private String nombre;
protected double peso;
```

```
private boolean sePuedeAsegurar(int currentYear, int edadMaxima){
    return this.calcularYears(currentYear)<=edadMaxima;
}
```

```
public int calcularYears(int currentYear){
    var retorno=1000;
    retorno=currentYear-this.year;
    return retorno;
}
```


Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Bases de POO para comprender patrones

Encapsulación

La encapsulación protege el estado interno de un objeto y controla la manera en que puede modificarse.

- Los atributos permanecen privados.
- Los métodos controlan el acceso.
- Las validaciones protegen las reglas del dominio.
- El objeto evita estados inválidos.
- Los getters sirven para acceder al valor asignado a un atributo.
- Los setters permiten modificar un atributo de manera controlada y pueden incorporar validaciones.

```java
public class Persona {

  private int edad;

  public int getEdad() {
    return edad;
  }

  public void setEdad(int edad) {
    if (edad < 0) {
      throw new IllegalArgumentException("Edad inválida");
    }
    this.edad = edad;
  }
}
```


Este material es de autoría y uso académico exclusivo de Mauricio Ortiz Ochoa

Bases de POO para comprender patrones

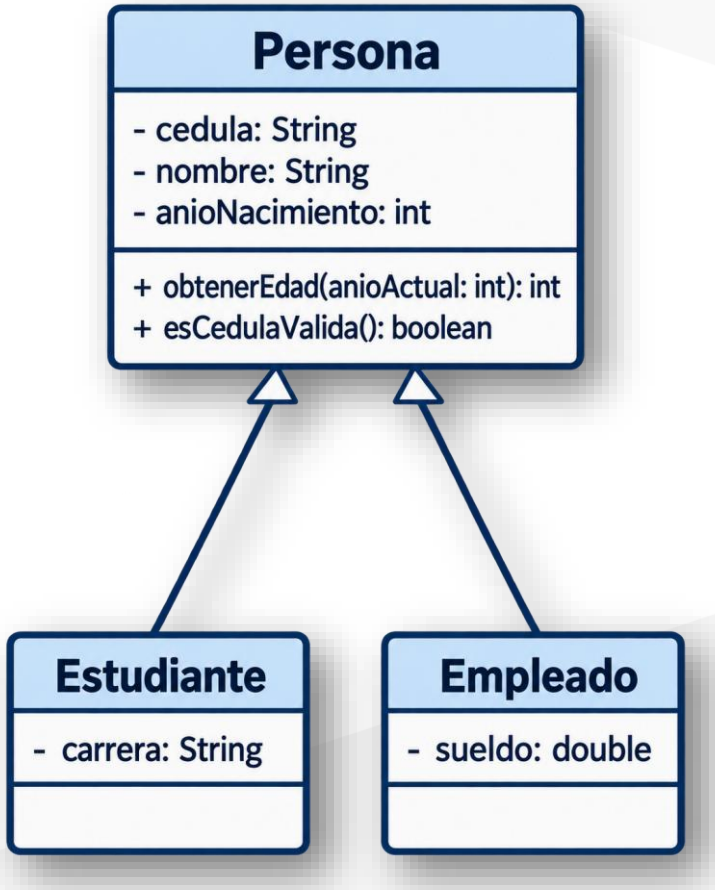
Herencia

- Permite la creación de clases a partir de otras ya existentes.
- Representa una relación «es un»: una subclase debe poder utilizarse donde se espera una instancia de la superclase.
- Debe existir una clase padre (superclase) y clases hijas (clases derivadas)
- Las clases derivadas heredan los métodos y atributos de la clase superclase.
- Ejemplo:
    - Tenemos una superclase Persona y dos clases derivadas Empleado y Estudiante
    - Los empleados y los estudiantes tienen atributos o métodos comunes y para optimizar el diseño, se puede crear una superclase que contenga todo lo común.
    - Los atributos o métodos que no sean comunes se mantendrán en las respectivas clases derivadas.

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Bases de POO para comprender patrones

Herencia




```java
public class Empleado extends Persona {
    private double sueldo;

    public double getSueldo() {
        return sueldo;
    }

    public void setSueldo(double sueldo) {
        if (sueldo < 0) {
            throw new IllegalArgumentException("Sueldo inválido");
        }
        this.sueldo = sueldo;
    }
}
```


Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Tema:

Actividad diagnóstica

- Propósito y organización de la asignatura
- Fundamentos de patrones
- Bases de POO para comprender patrones
- *Actividad diagnóstica*

Este material es de autoría y uso académico exclusivo de Mauricio Ortiz Ochoa

Actividad diagnóstica

Diseñen un sistema de pedidos

Una tienda necesita gestionar clientes, productos, pedidos y pagos:

- Un cliente puede realizar varios pedidos.
- Cada pedido pertenece a un solo cliente.
- Un pedido contiene uno o más productos y registra sus cantidades.
- El pedido calcula su total y controla los cambios de estado.
- El pago puede realizarse mediante tarjeta o transferencia.

Elaboren colaborativamente un modelo UML que represente este dominio.

Actividad en grupo · 40 minutos · No calificable · IA permitida

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Actividad diagnóstica

Entregable y uso de IA

Cada grupo debe entregar

- Un diagrama con clases, atributos y operaciones.
- Visibilidad, encapsulación y validaciones.
- Asociaciones con multiplicidad y navegabilidad.
- Una relación de herencia donde resulte pertinente.
- Código Java o pseudocódigo de una operación con validación.

Uso crítico de IA

- Acuerden y documenten primero una propuesta como grupo.
- Utilicen IA para revisar errores y posibles mejoras.
- Registren el prompt principal.
- Identifiquen una recomendación aceptada y una rechazada o modificada.
- Justifiquen las decisiones del grupo.
- Entreguen un único archivo por grupo con el diagrama, el código y la evidencia del uso de IA.

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Unidad 01.01

Retroalimentación y cierre

Qué vimos hoy

- Los patrones aparecen en diferentes dominios como respuestas recurrentes a problemas similares.
- Un patrón documenta un contexto, un problema, las fuerzas que deben equilibrarse, una solución y sus consecuencias.
- Los patrones GoF ofrecen un vocabulario compartido y se clasifican en creacionales, estructurales y de comportamiento.
- La POO proporciona los elementos para implementarlos: abstracción, objetos, clases, encapsulación y herencia.
- UML permite comunicar clases y relaciones mediante asociaciones, multiplicidades, navegabilidad y dependencias.
- La actividad diagnóstica permitió reconocer nuestras fortalezas y aspectos por reforzar, utilizando IA con criterio crítico.

Antes de finalizar: ¿Qué diferencia existe entre copiar una solución y aplicar correctamente un patrón?

Idea final: Un patrón no es una receta ni un fragmento de código: es una decisión de diseño reutilizable que solo tiene sentido dentro de un contexto.

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Anexos

Bibliografía

Gamma, E., Helm, R., Johnson, R. y Vlissides, J.; Design Patterns: Elements of Reusable Object-Oriented Software; 1.ª edición, 1994.
Sarcar, V.; Java Design Patterns: A Hands-On Experience with Real-World Examples; 3.ª edición, 2022.
Musch, O.; Design Patterns with Java: An Introduction; 1.ª edición, 2023.
Fowler, M.; UML Distilled: A Brief Guide to the Standard Object Modeling Language; 3.ª edición, 2004.
Shvets, A.; Sumérgete en los patrones de diseño; 1.ª edición, 2019.
IEEE Computer Society; Guide to the Software Engineering Body of Knowledge — SWEBOK Guide V4.0a; versión 4.0a, 2025.
Asaad, J. y Avksentieva, E.; Assessing the Impact of GoF Design Patterns on Software Engineering Practices; 2025.

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Anexos

Links de Referencia

- http://www.vincehuston.org/dp/
- https://www.campusmvp.es/recursos/post/los-conceptos-fundamentales-sobre-programacion-orientada-objetos-explicados-de-manera-simple.aspx
- https://codideep.com/blogpost/stupid-vs-solid
- https://refactoring.guru/design-patterns
- https://reactiveprogramming.io/books/design-patterns/es

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Anexos

Herramientas usadas en el curso

- Java SE 11 o superior
  - Entorno de desarrollo para la creación de aplicaciones utilizando el lenguaje de programación Java.
  - https://www.oracle.com/java/technologies/java-se-glance.html
- Apache Netbeans
  - Entorno integrado de desarrollo (IDE)
  - Resalta el código fuente sintáctica y semánticamente,
  - Permite refactorizar fácilmente el código con una gama de herramientas útiles y potentes.

    https://netbeans.apache.org/download/index.html
  - Plugins

    https://plugins.netbeans.apache.org/

    Rainbow Braces

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Anexos

Cuentas necesarias

Google Drive

- Servirá para almacenar los diagramas en la nube
- https://drive.google.com/drive

Diagrams.net Draw.io

- Permite la creación en línea de diagramas de clase UML
- https://app.diagrams.net/

GitHub

- Permite el almacenamiento y descarga de proyectos de programación en la nube
- https://github.com/

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Gracias por su atención

CC BY NC ND